Pipelined Processors

Dr. V S M Srinivasavarma

Outline

- Pipelining
- Hazards
  - Structural
  - Data
  - Control

Overview

- Pipelining is widely used in modern processors
- Pipelining improves systems performance in terms of throughput
- Pipelined organization requires sophisticated compilation techniques

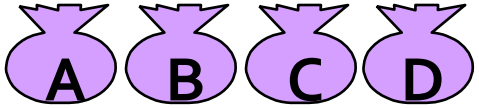
Basic Concepts

Making the Execution of Programs Faster

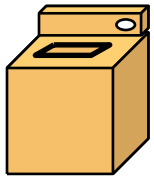
- Use faster circuit technology to build the processor and the main memory.
- Arrange the hardware so that more than one operation can be performed at the same time.
- In the latter way, the number of operations performed per second is increased even though the elapsed time needed to perform any one operation is not changed.

Traditional Pipeline Concept

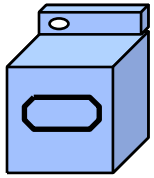
- Laundry Example
- Ann, Brian, Cathy, Dave each have one load of clothes to wash, dry, and fold



- Washer takes 30 minutes

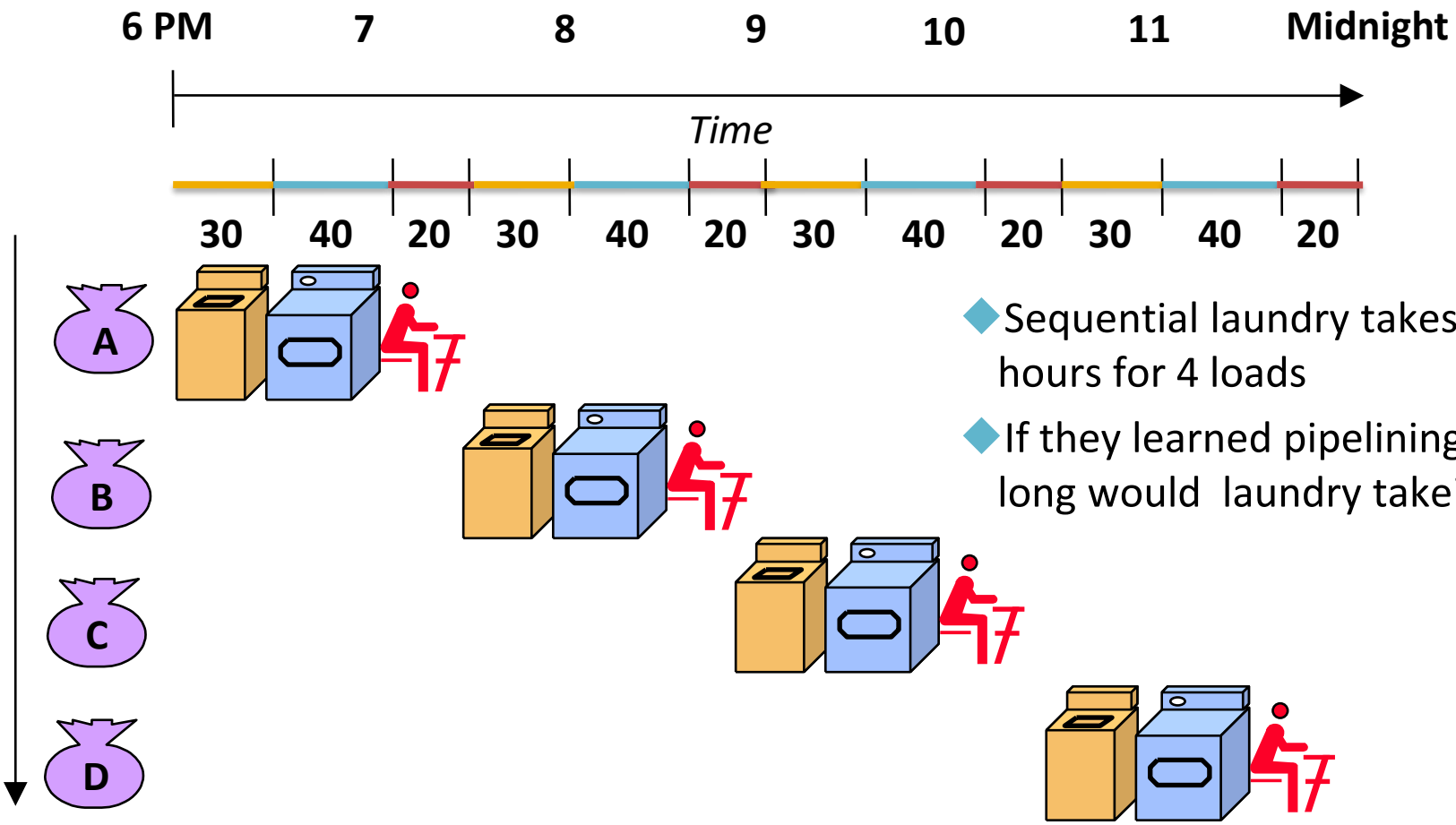


- Dryer takes 40 minutes



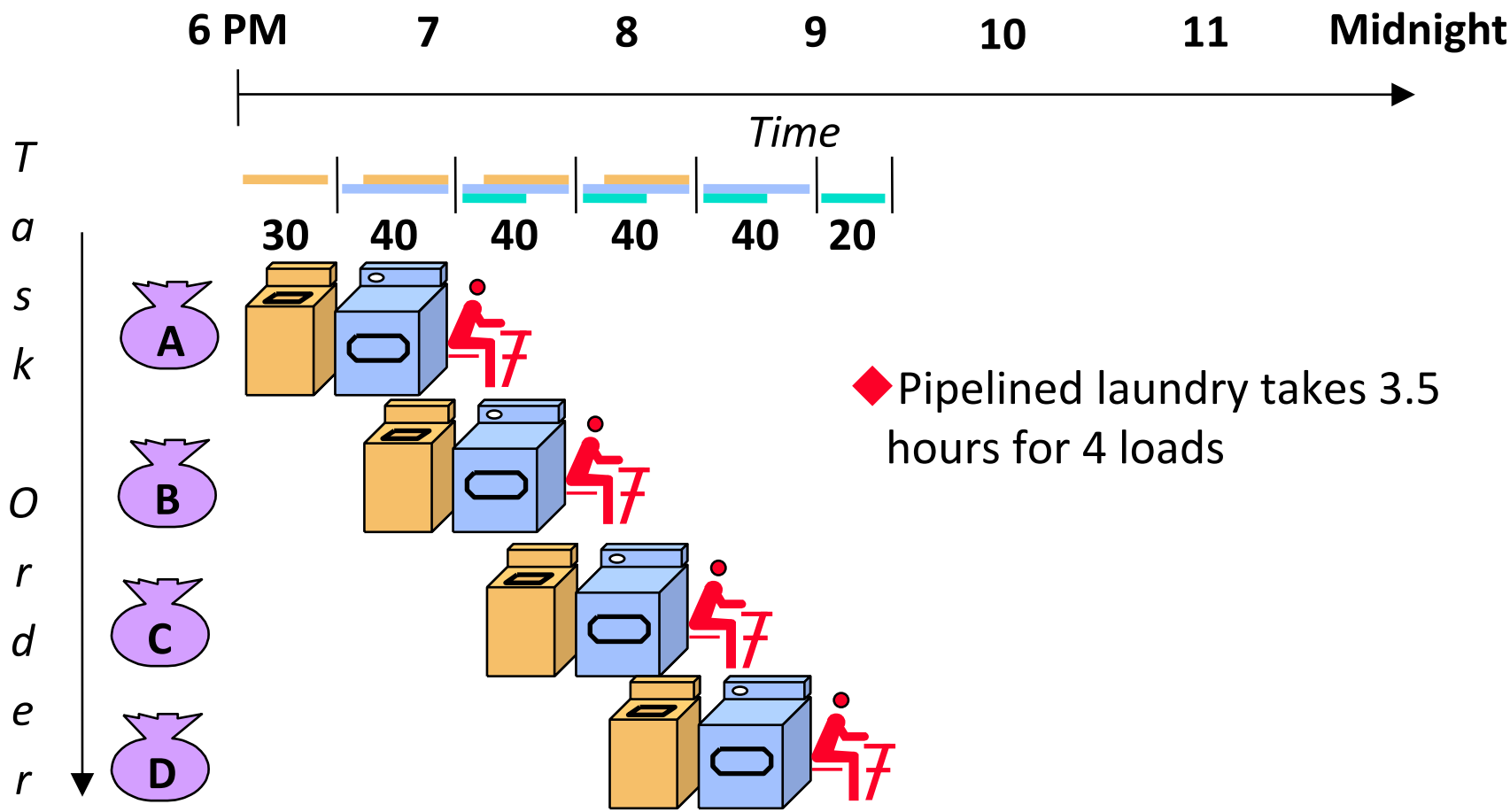
- "Folder" takes 20 minutes

Traditional Pipeline Concept



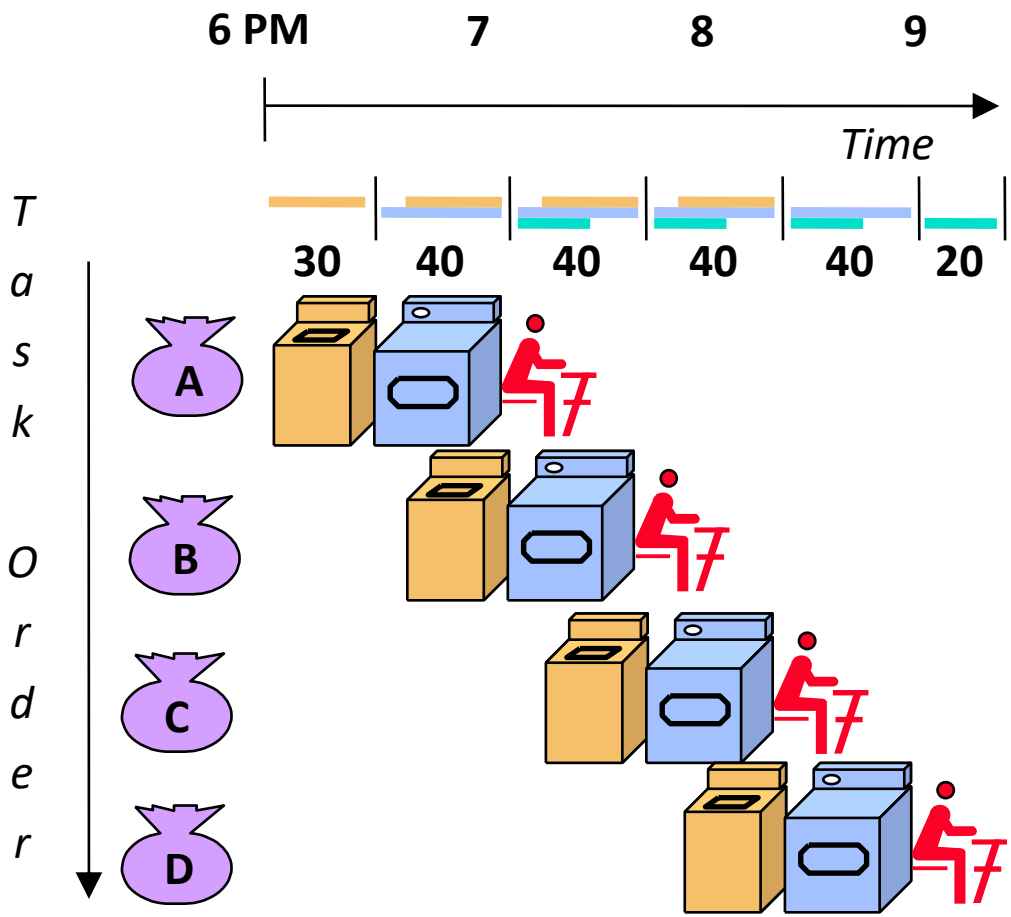
- Sequential laundry takes 6 hours for 4 loads
- If they learned pipelining, how long would laundry take?

Traditional Pipeline Concept



- Pipelined laundry takes 3.5 hours for 4 loads

Traditional Pipeline Concept



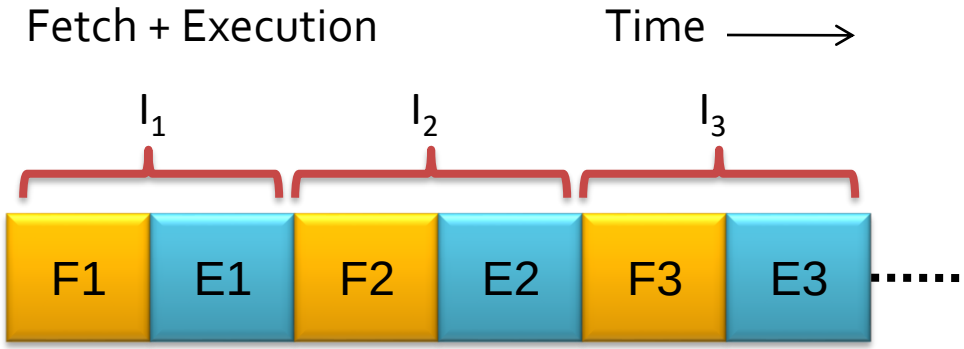
Traditional Pipeline Concept

- Pipelining doesn't help latency of single task, it helps throughput of entire workload
- Pipeline rate limited by slowest pipeline stage
- Multiple tasks operating simultaneously using different resources
- Potential speedup = Number of pipe stages
- Unbalanced lengths of pipe stages reduces speedup
- Time to "fill" pipeline and time to "drain" it reduces speedup
- Stall for Dependences

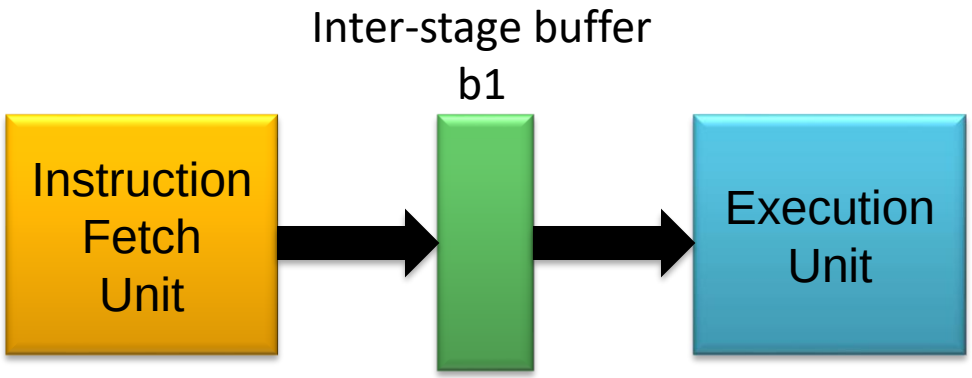
Pipelining

- A technique of decomposing a sequential process into sub operations, with each sub process being executed in a partial dedicated segment that operates concurrently with all other segments

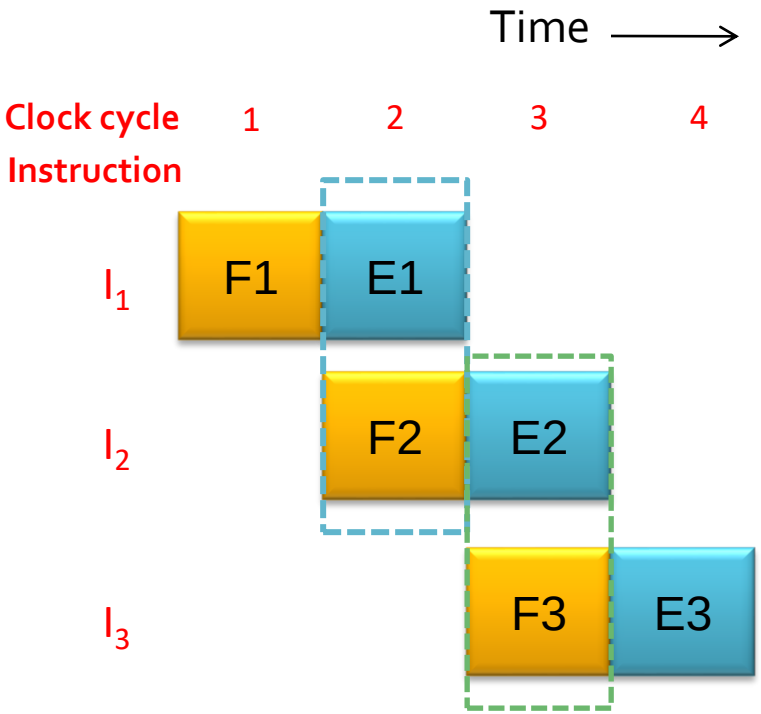
Use the Idea of Pipelining in a Computer



Sequential execution



Hardware Organization

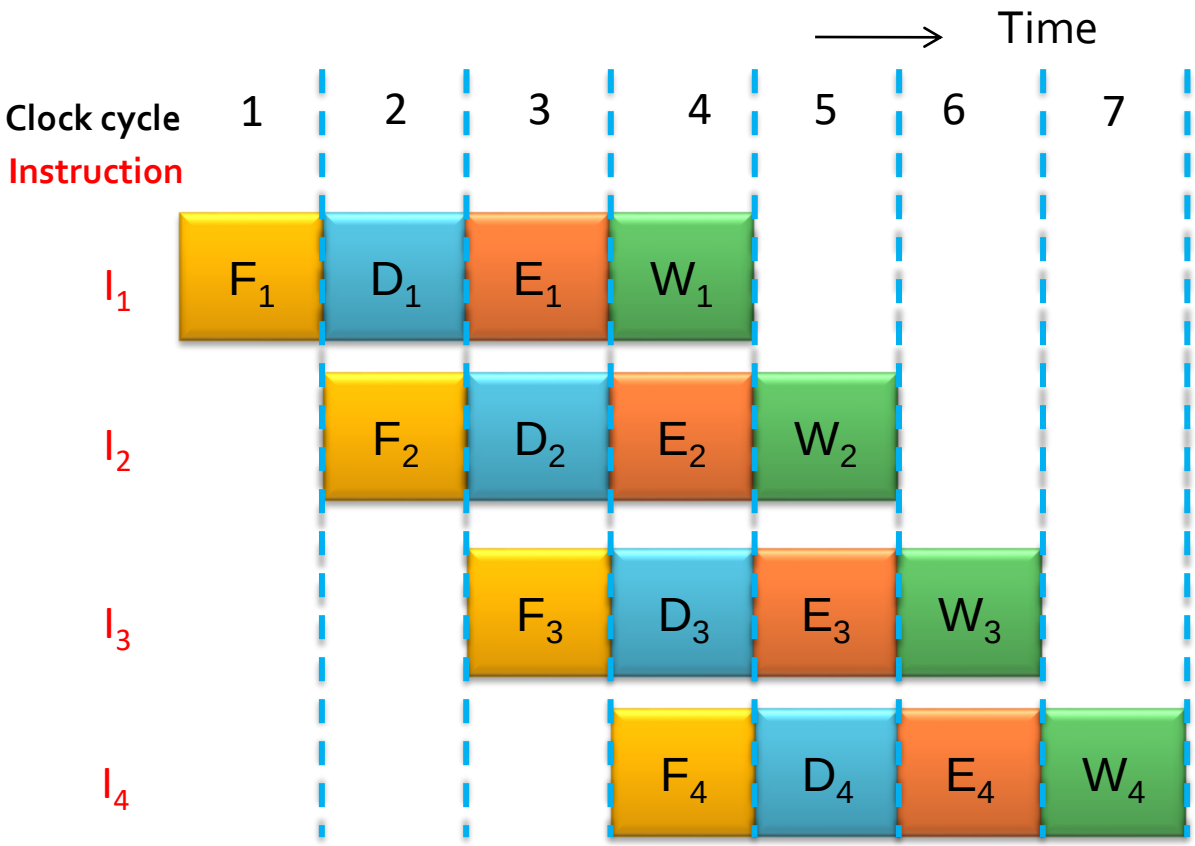


Pipelined Execution

Instruction Cycle

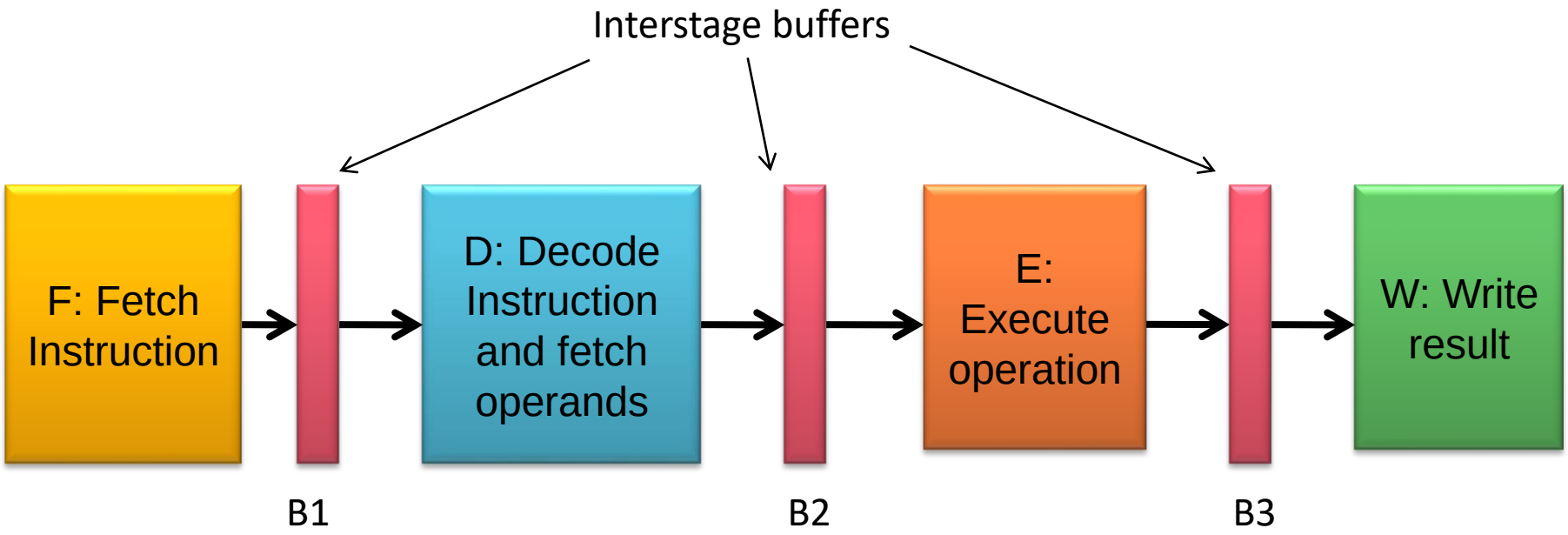
- The complete execution of instruction can be divided into four steps
  - Fetch: read the instruction from the memory
  - Decode: decode the instruction and fetch the source operand(s)
  - Execute: perform the operation specified by the instruction
  - Write: store the result in the destination location

Instruction Pipeline



Instruction Execution Divided into four steps

A 4-Stage Instruction Pipeline



Hardware Organization

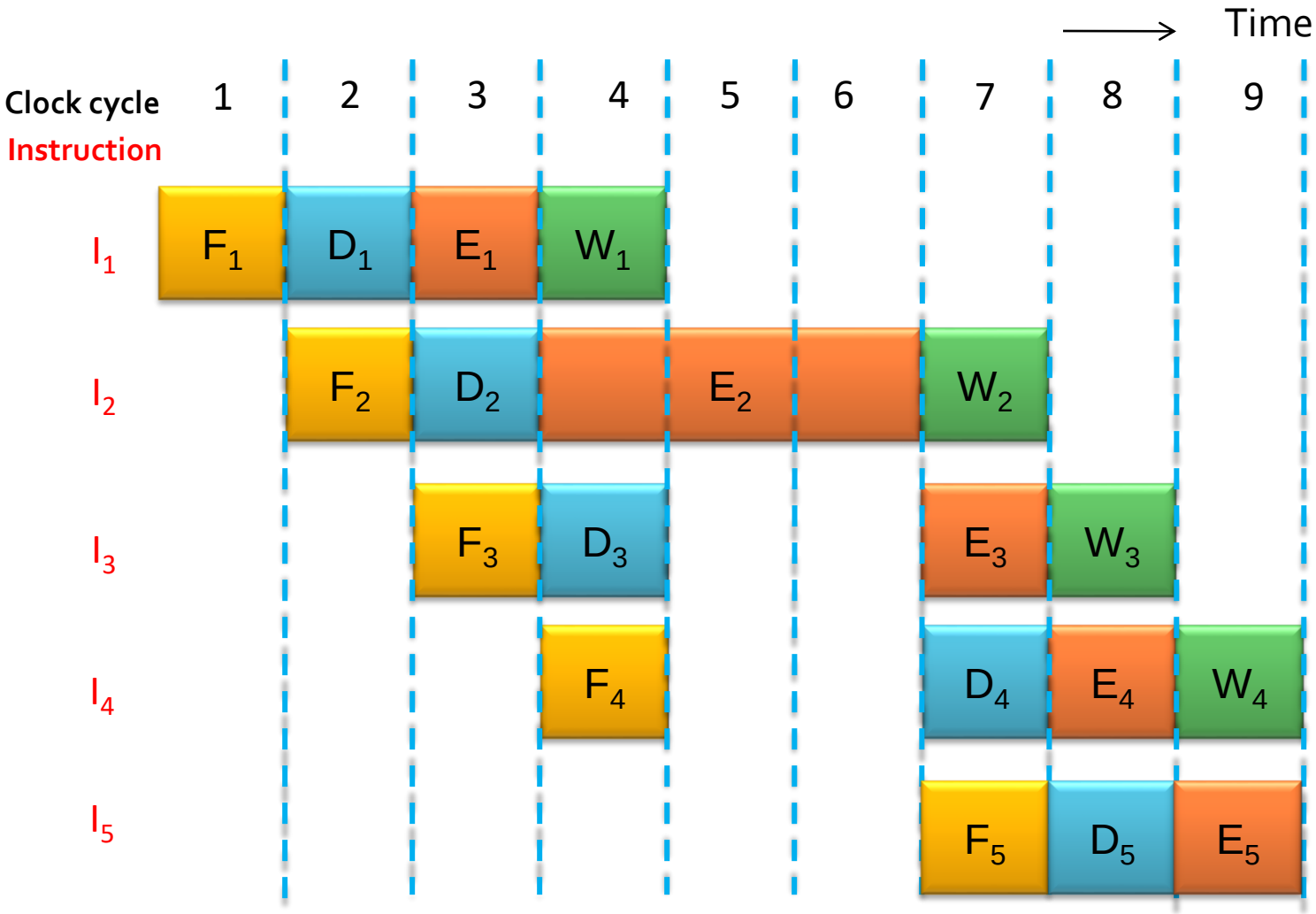
Role of Cache Memory

- Each pipeline stage is expected to complete in one clock cycle.
- The clock period should be long enough to let the slowest pipeline stage to complete.
- Faster stages can only wait for the slowest one to complete.
- Since main memory is very slow compared to the execution, if each instruction needs to be fetched from main memory, pipeline is almost useless.
- Fortunately, we have cache.

Pipeline Performance

- The potential increase in performance resulting from pipelining is proportional to the number of pipeline stages.
- However, this increase would be achieved only if all pipeline stages require the same time to complete, and there is no interruption throughout program execution.
- Unfortunately, this is NOT TRUE.

Pipeline Performance



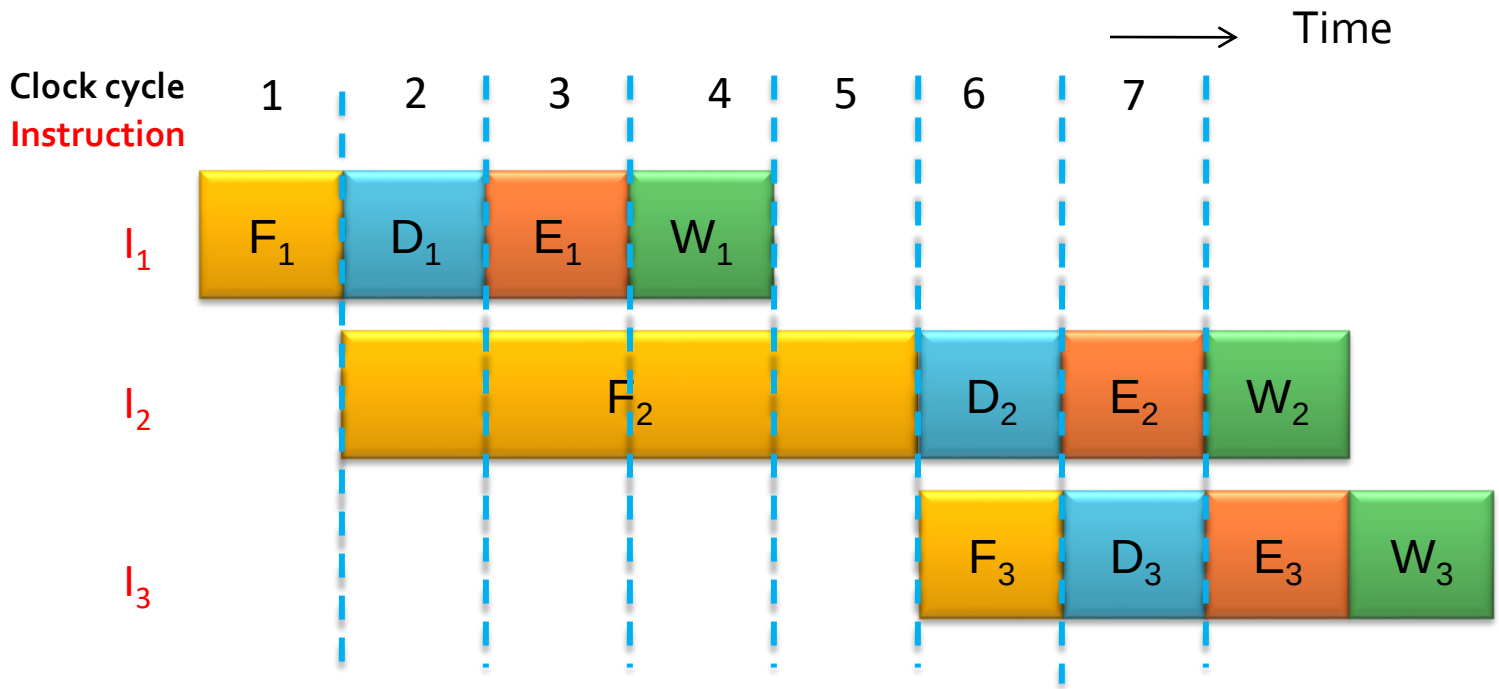
Effect of an execution operation taking more than one clock cycle

Pipeline Performance

- The previous pipeline is said to have been stalled for two clock cycles.
- Any condition that causes a pipeline to stall is called a **hazard**.
- **Data hazard** – any condition in which either the source or the destination operands of an instruction are not available at the time expected in the pipeline. So some operation has to be delayed, and the pipeline stalls.
- **Instruction (control) hazard** – a delay in the availability of an instruction causes the pipeline to stall.
- **Structural hazard** – the situation when two instructions require the use of a given hardware resource at the same time

Pipeline Performance

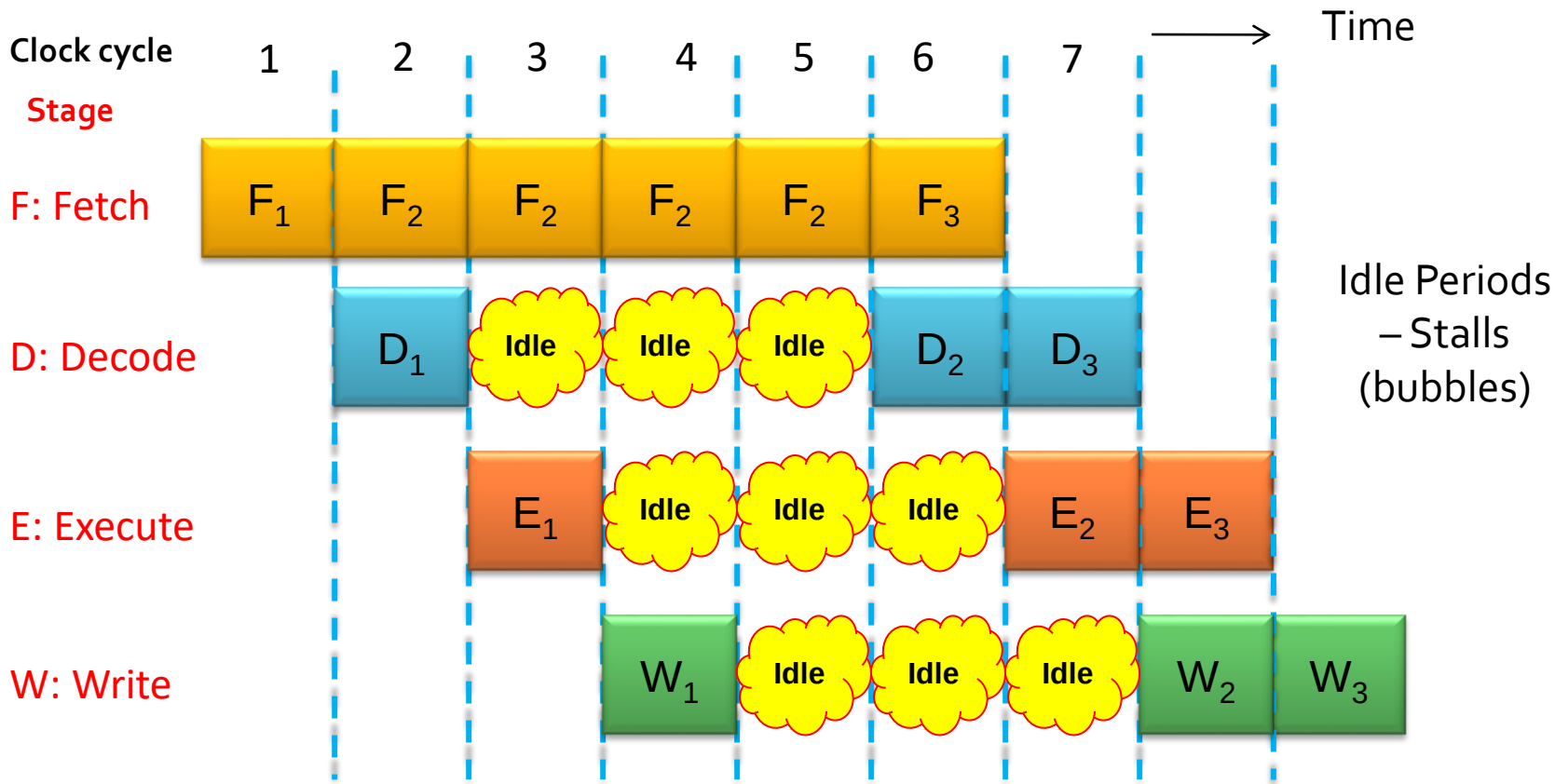
Instruction Hazard – Pipeline stall caused by a Cache miss in F2



Instruction Execution steps in successive clock cycles

Pipeline Performance

Instruction Hazard – Pipeline stall caused by a Cache miss in F2



Function performed by each processor stage in successive clock cycles

Pipeline Performance

- Again, pipelining does not result in individual instructions being executed faster; rather, it is the throughput that increases.
- Throughput is measured by the rate at which instruction execution is completed.
- Pipeline stall causes degradation in pipeline performance.
- We need to identify all hazards that may cause the pipeline to stall and to find ways to minimize their impact.

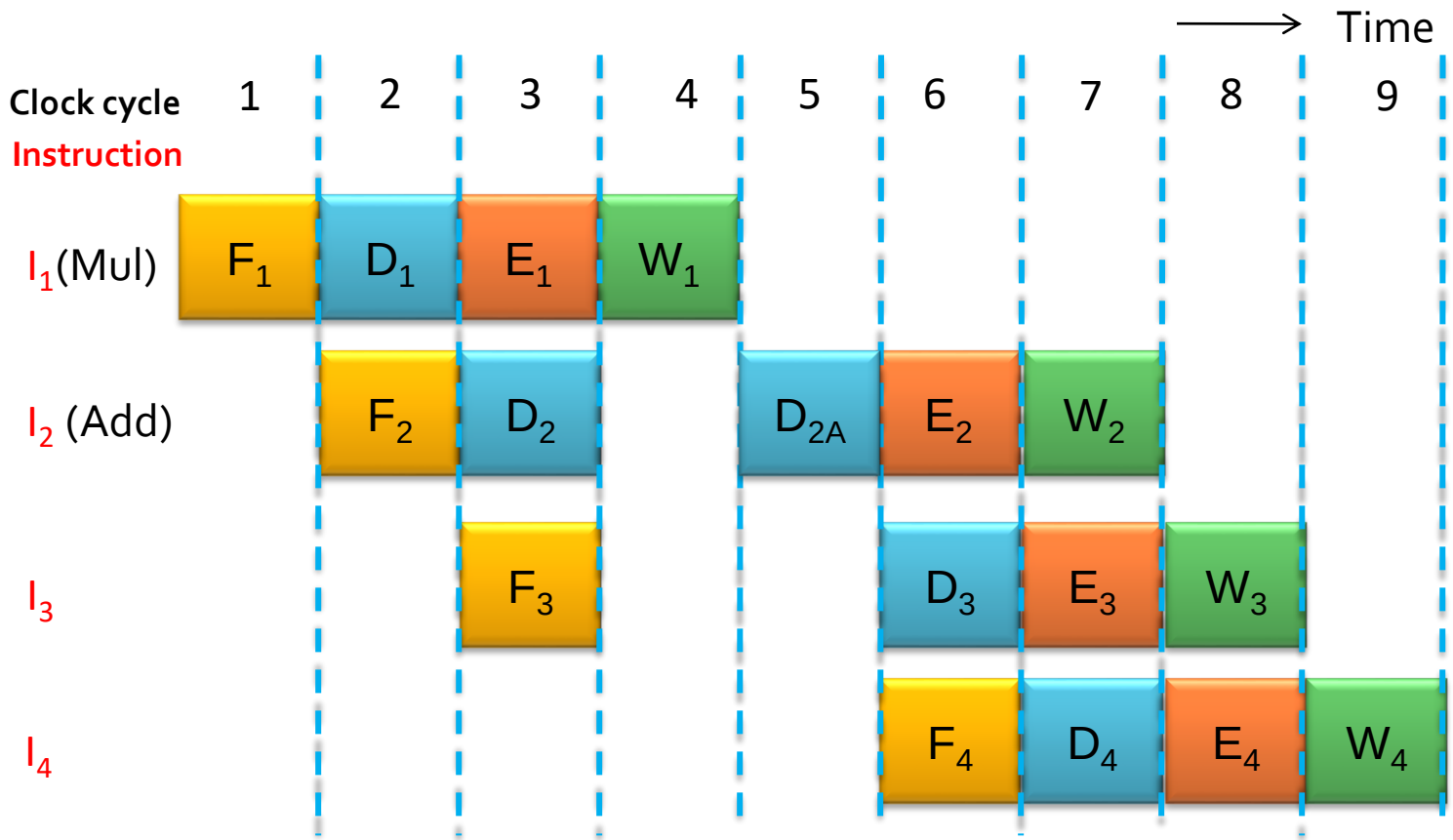
Data Hazards

- We must ensure that the results obtained when instructions are executed in a pipelined processor are identical to those obtained when the same instructions are executed sequentially.
- Hazard occur
  - $A \leftarrow 3 + A$
  - $B \leftarrow 4 \times A$
- No hazard
  - $A \leftarrow 5 \times C$
  - $B \leftarrow 20 + C$
- When two operations depend on each other, they must be executed sequentially in the correct order.
- Another example:

```
Mul  R2, R3, R4        // R4 = R2 * R3
Add  R5, R4, R6        // R6 =  R5 + R4
```

Data Hazards

Data Hazard – Mul R2, R3, R4
Add R5, R4, R6



Pipeline stalled by data dependency between D_2 and W_1

Read After Write (RAW)

$Instr_J$ tries to read operand before $Instr_I$ writes it

```
I: add r1,r2,r3
J: sub r4,r1,r3
```

Execution Order is:
$Instr_I$
$Instr_J$

- Caused by a "Dependence" (in compiler nomenclature). This hazard results from an actual need for communication.

Write After Read (WAR)

$Instr_J$ tries to write operand *before* $Instr_I$ reads i

- Gets wrong operand

```
I: sub r4,r1,r3
J: add r1,r2,r3
K: mul r6,r1,r7
```

Execution Order is:
$Instr_I$
$Instr_J$

- Called an "anti-dependence" by compiler writers. This results from reuse of the name "r1".

◆ Can't happen in MIPS 5 stage pipeline because:

- All instructions take 5 stages, and
- Reads are always in stage 2, and
- Writes are always in stage 5

Write After Write (WAW)

$Instr_J$ tries to write operand *before* $Instr_I$ writes it

- Leaves wrong result ($Instr_I$ not $Instr_J$)

```
I: sub r1,r4,r3
J: add r1,r2,r3
K: mul r6,r1,r7
```

Execution Order is:
$Instr_I$
$Instr_J$

- Called an "output dependence" by compiler writers
  This also results from the reuse of name "r1".

- Can't happen in MIPS 5 stage pipeline because:
  - All instructions take 5 stages, and
  - Writes are always in stage 5

- Will see WAR and WAW in later more complicated pipes

Data Hazards

Simple Solution to RAW

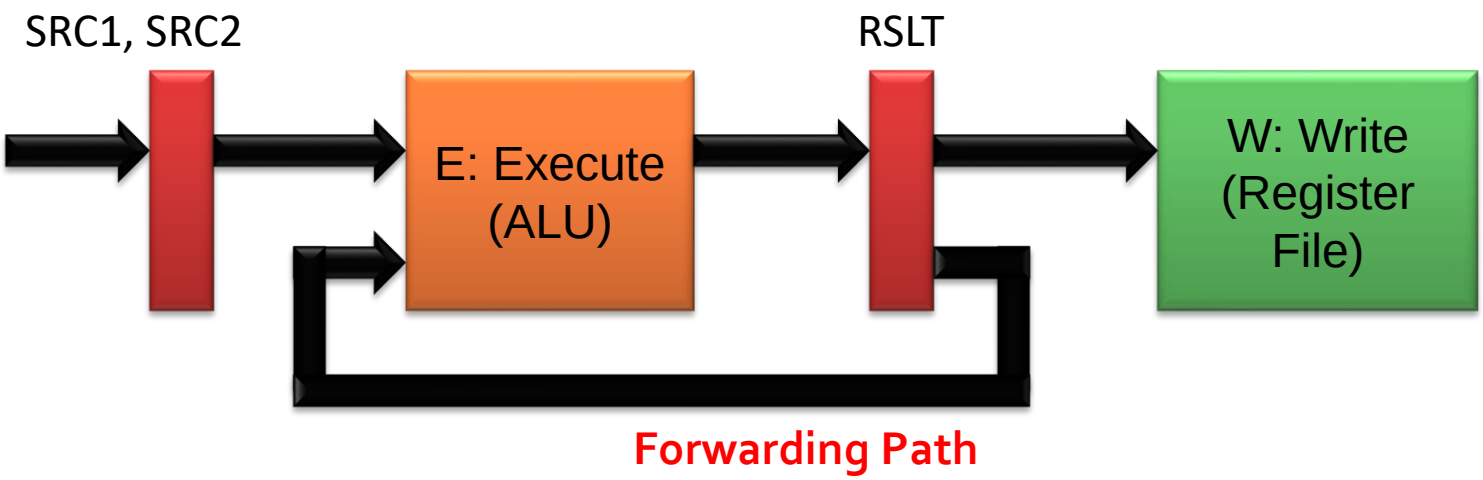
- Hardware detects RAW and stalls
- Assumes register written then read each cycle
  - + low cost to implement, simple
  - -- reduces IPC
- Try to minimize stalls

Minimizing RAW stalls

- Bypass/forward/shortcircuit (We will use the word “forward”)
- Use data before it is in the register
  - + reduces/avoids stalls
  - -- complex
- Crucial for common RAW hazards

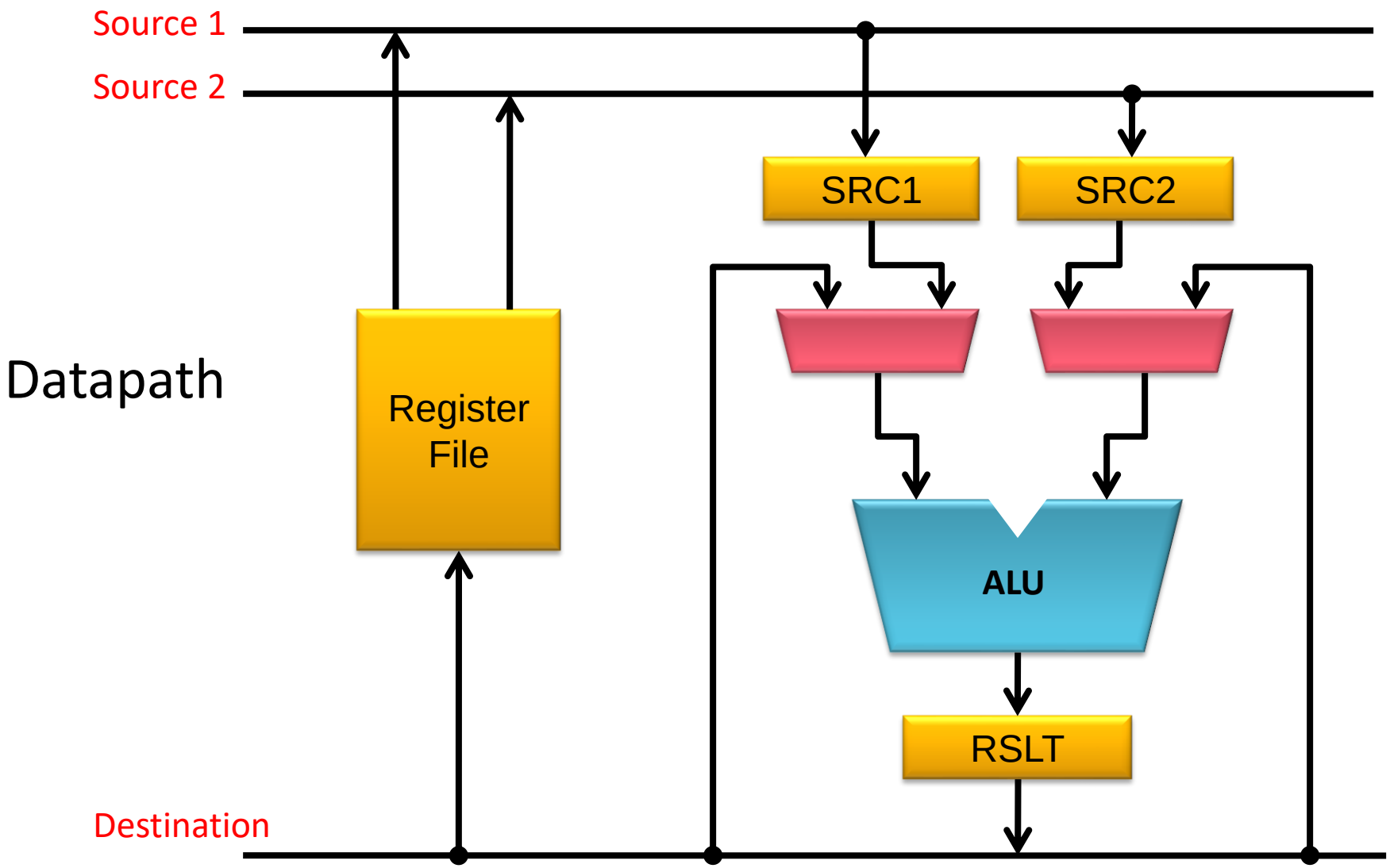
Operand Forwarding

- Instead of from the register file, the second instruction can get data directly from the output of ALU after the previous instruction is completed.
- A special arrangement needs to be made to “forward” the output of ALU to the input of ALU



Position of the source and result registers in the processor pipeline

Operand Forwarding Hardware



Handling Data Hazards in Software

- Let the compiler detect and handle the hazard:

  I1: Mul R2, R3, R4

  NOP

  NOP

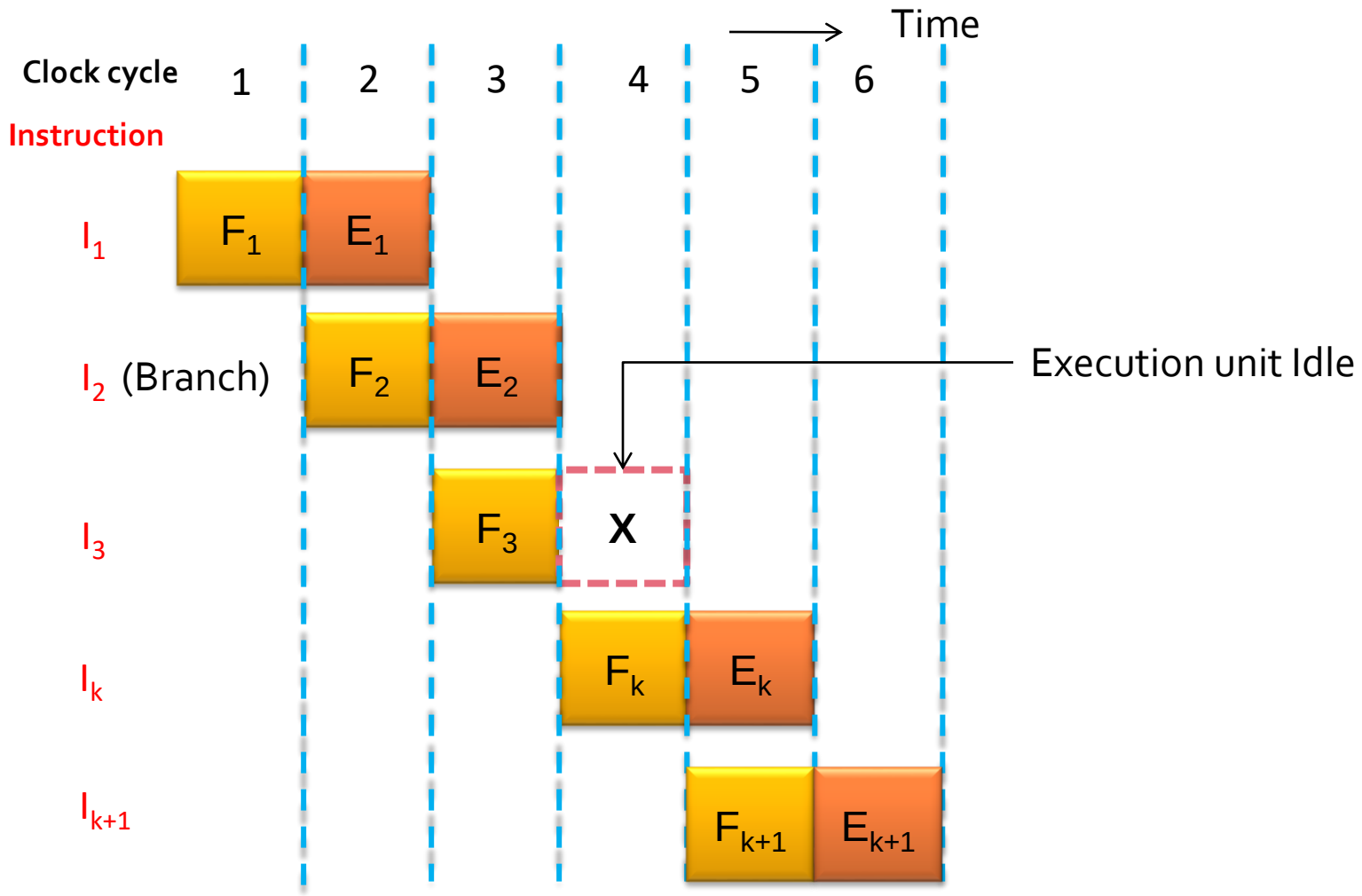
  I2: Add R5, R4, R6

- The compiler can reorder the instructions to perform some useful work during the NOP slots.

Instruction Hazards

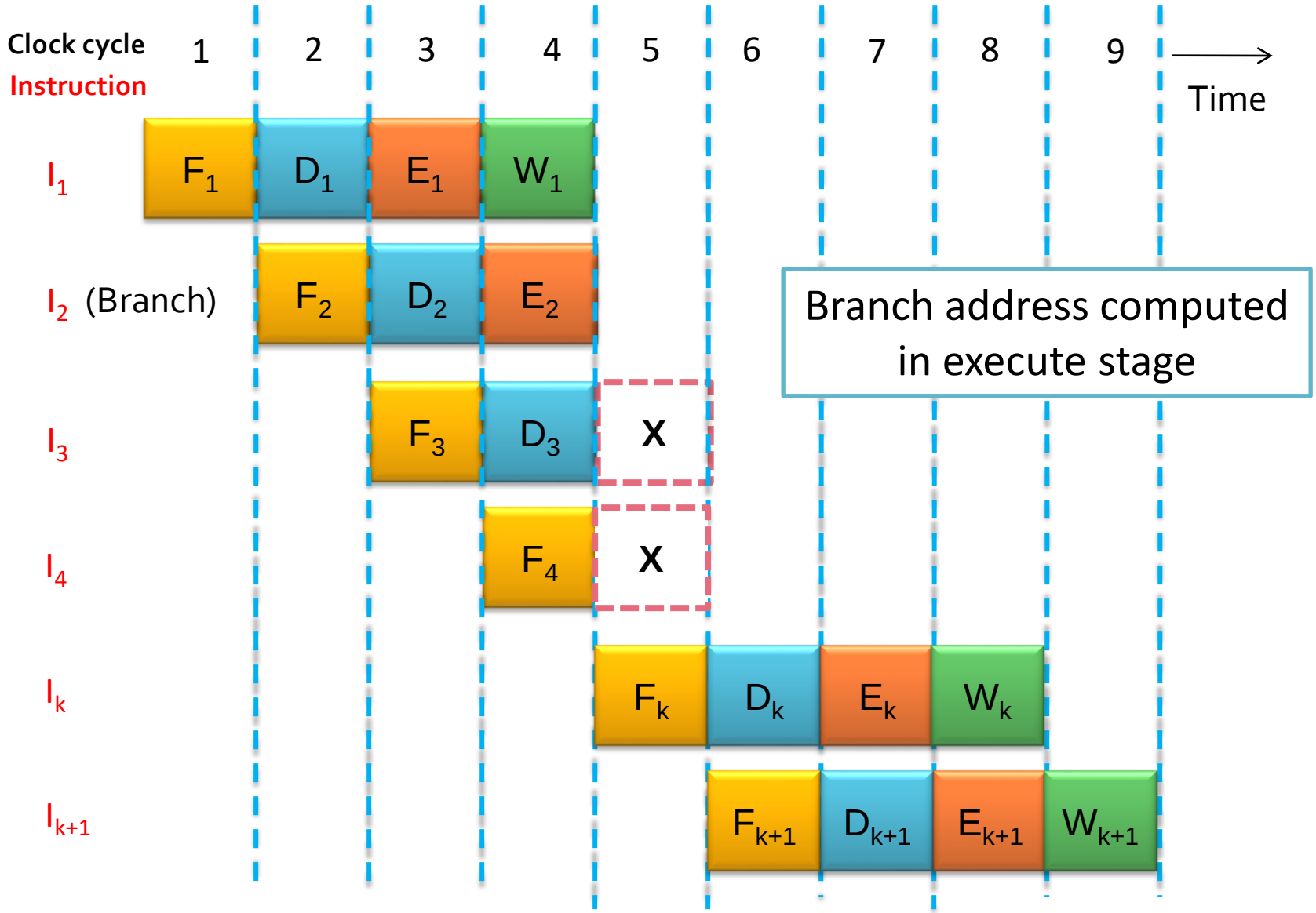
- Whenever the stream of instructions supplied by the instruction fetch unit is interrupted, the pipeline stalls.
  - Cache miss
  - Branch

Unconditional Branches

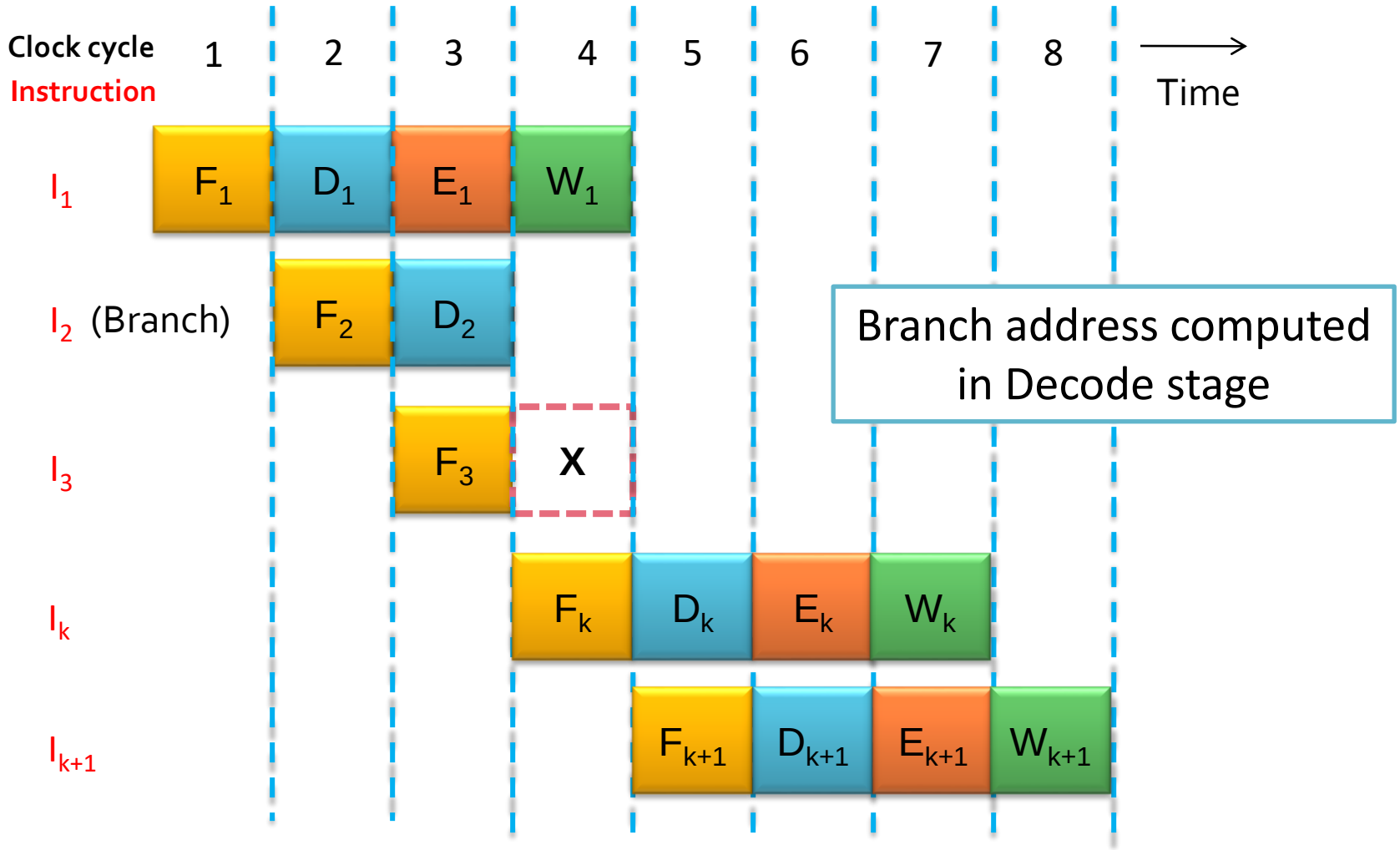


An Idle cycle caused by a branch instruction

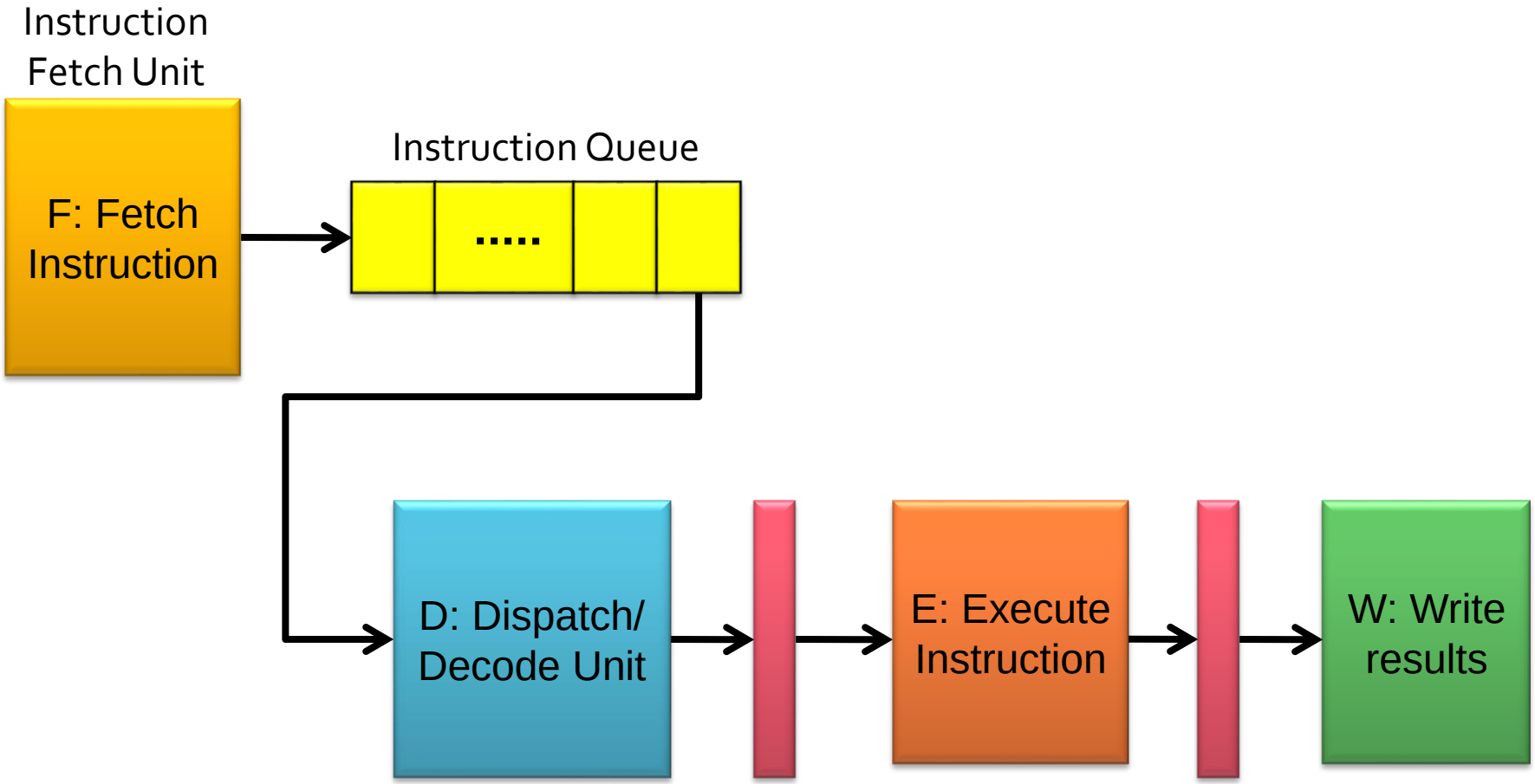
Branch Timing



Branch Timing



Instruction Queue and Prefetching



Use of Instruction Queue in the Hardware Organization

Branch Folding

- Branch folding – executing the branch instruction concurrently with the execution of other instructions.
- Branch folding occurs only if at the time a branch instruction is encountered, at least one instruction is available in the queue other than the branch instruction.
- Therefore, it is desirable to arrange for the queue to be full most of the time, to ensure an adequate supply of instructions for processing.
- This can be achieved by increasing the rate at which the fetch unit reads instructions from the cache.
- Having an instruction queue is also beneficial in dealing with cache misses.

Conditional Braches

- A conditional branch instruction introduces the added hazard caused by the dependency of the branch condition on the result of a preceding instruction.
- The decision to branch cannot be made until the execution of that instruction has been completed.
- Branch instructions represent about 20% of the dynamic instruction count of most programs.

Delayed Branch

- The instructions in the delay slots are always fetched. Therefore, we would like to arrange for them to be fully executed whether or not the branch is taken.
- The objective is to place useful instructions in these slots.
- The effectiveness of the delayed branch approach depends on how often it is possible to reorder instructions.

Delayed Branch

LOOP	Shift_left	R1
	Decrement	R2
	Branch =0	LOOP
NEXT	Add	R1, R3

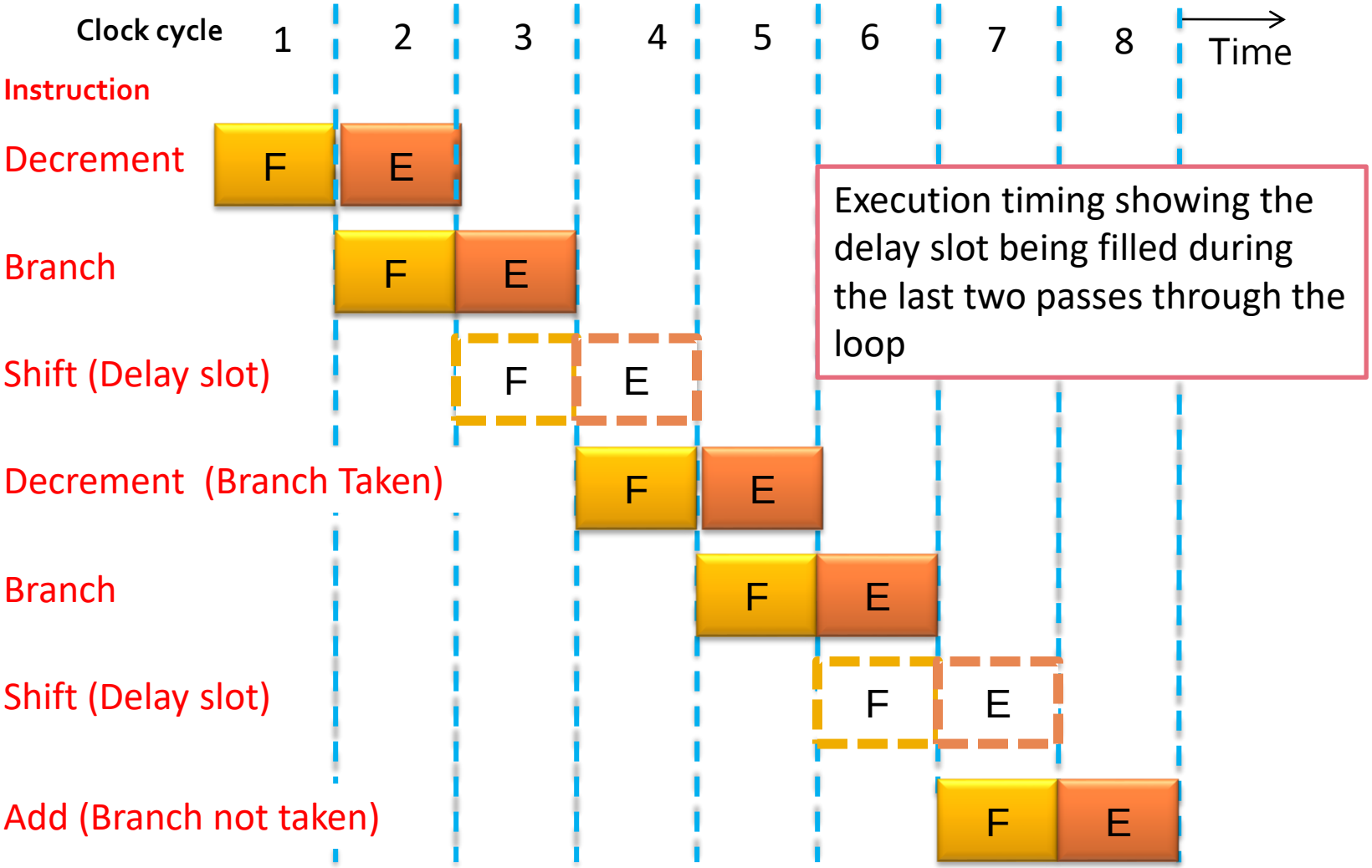
Original Program Loop

LOOP	Decrement	R2
	Branch =0	LOOP
	Shift_left	R1
NEXT	Add	R1, R3

Reordered Instructions

Reordering of instructions for a delayed branch

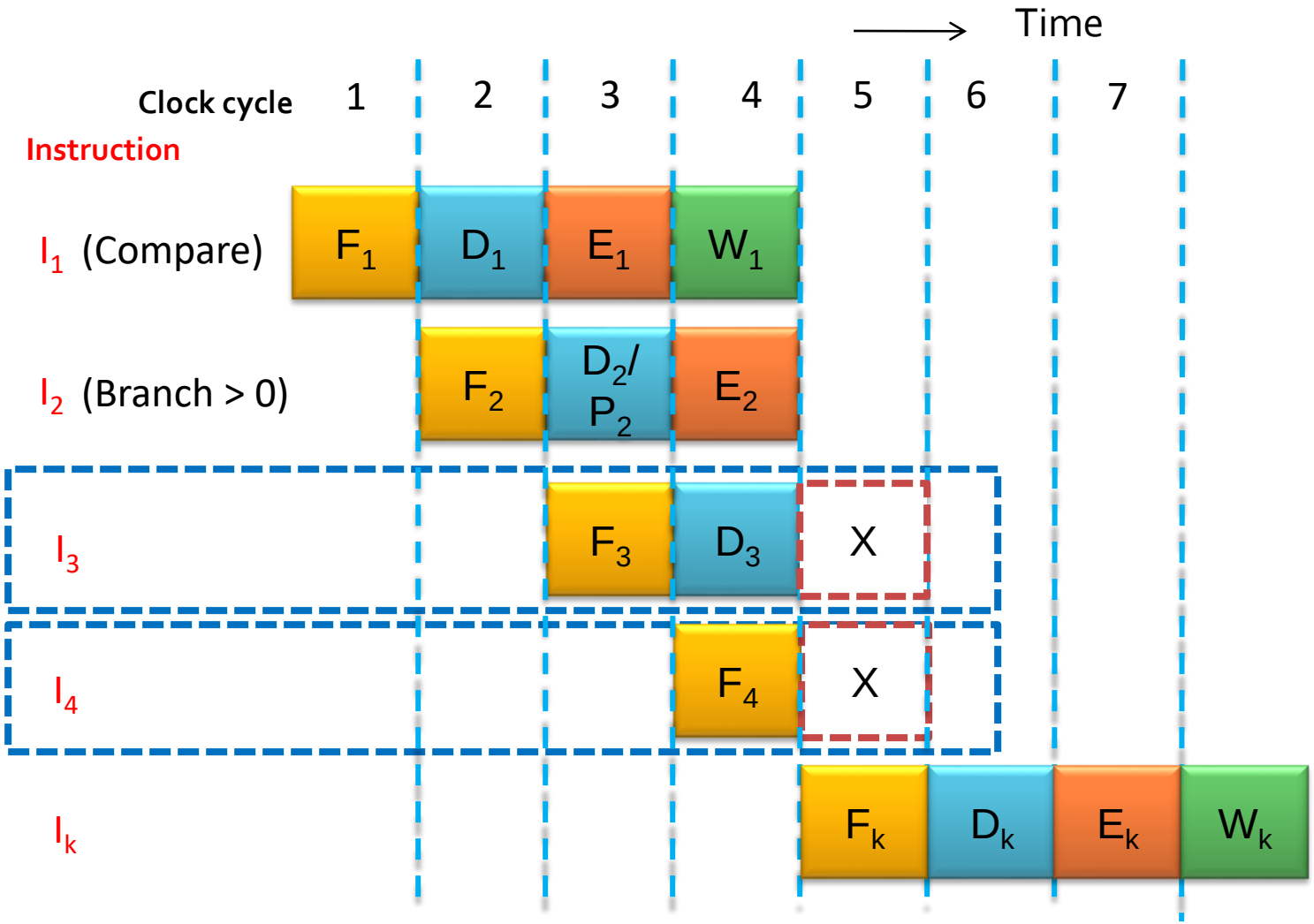
Delayed Branch



Branch Prediction

- To predict whether or not a particular branch will be taken.
- Simplest form: assume branch will not take place and continue to fetch instructions in sequential address order.
- Until the branch is evaluated, instruction execution along the predicted path must be done on a speculative basis.
- Speculative execution: instructions are executed before the processor is certain that they are in the correct execution sequence.
- Need to be careful so that no processor registers or memory locations are updated until it is confirmed that these instructions should indeed be executed.

Incorrectly Predicted Branch



Timing when a branch decision has been incorrectly predicted as not taken.

Branch Prediction

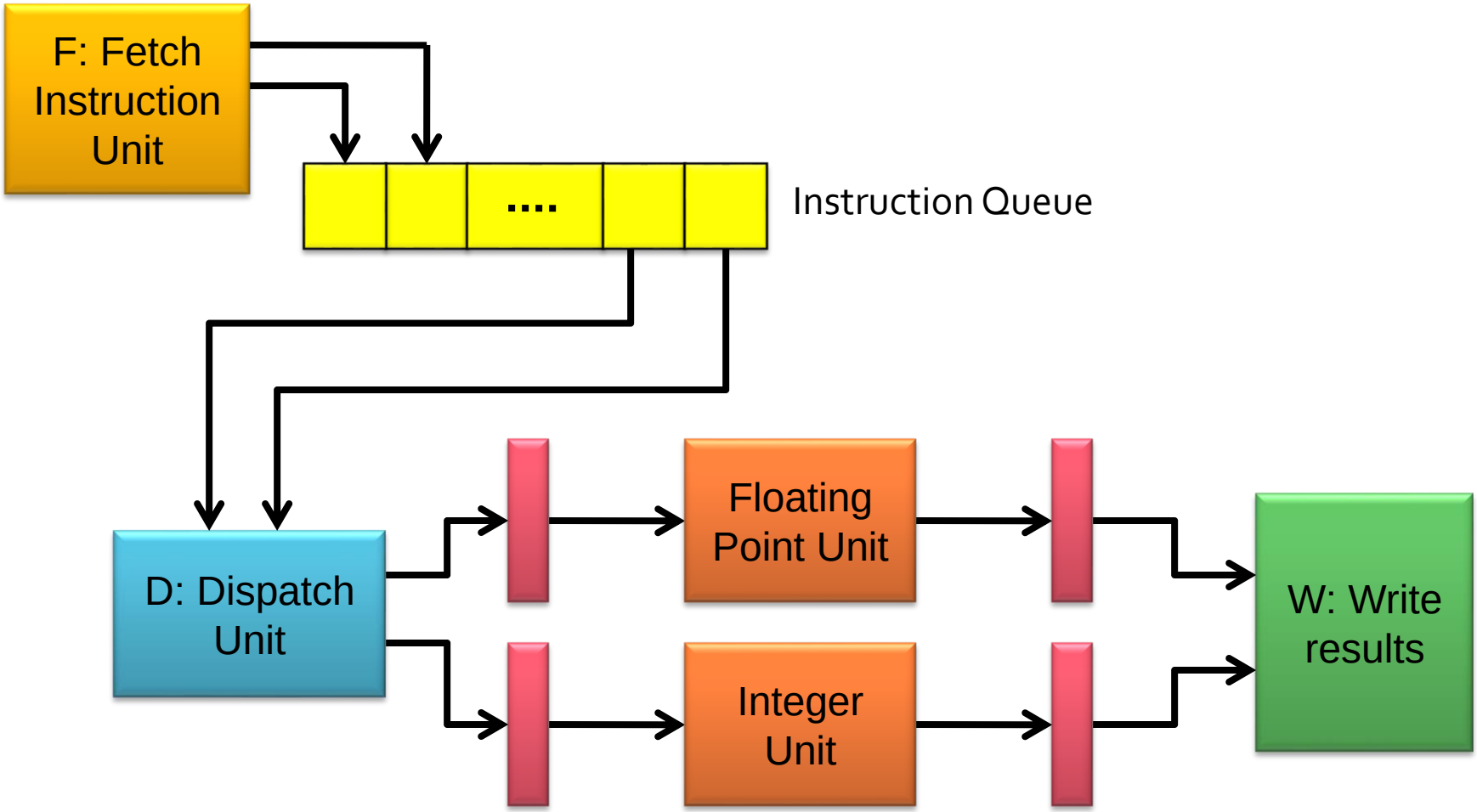
- Better performance can be achieved if we arrange for some branch instructions to be predicted as taken and others as not taken.
- Use hardware to observe whether the target address is lower or higher than that of the branch instruction.
- Let compiler include a branch prediction bit.
- So far the branch prediction decision is always the same every time a given instruction is executed – static branch prediction.

Superscalar Operation

Overview

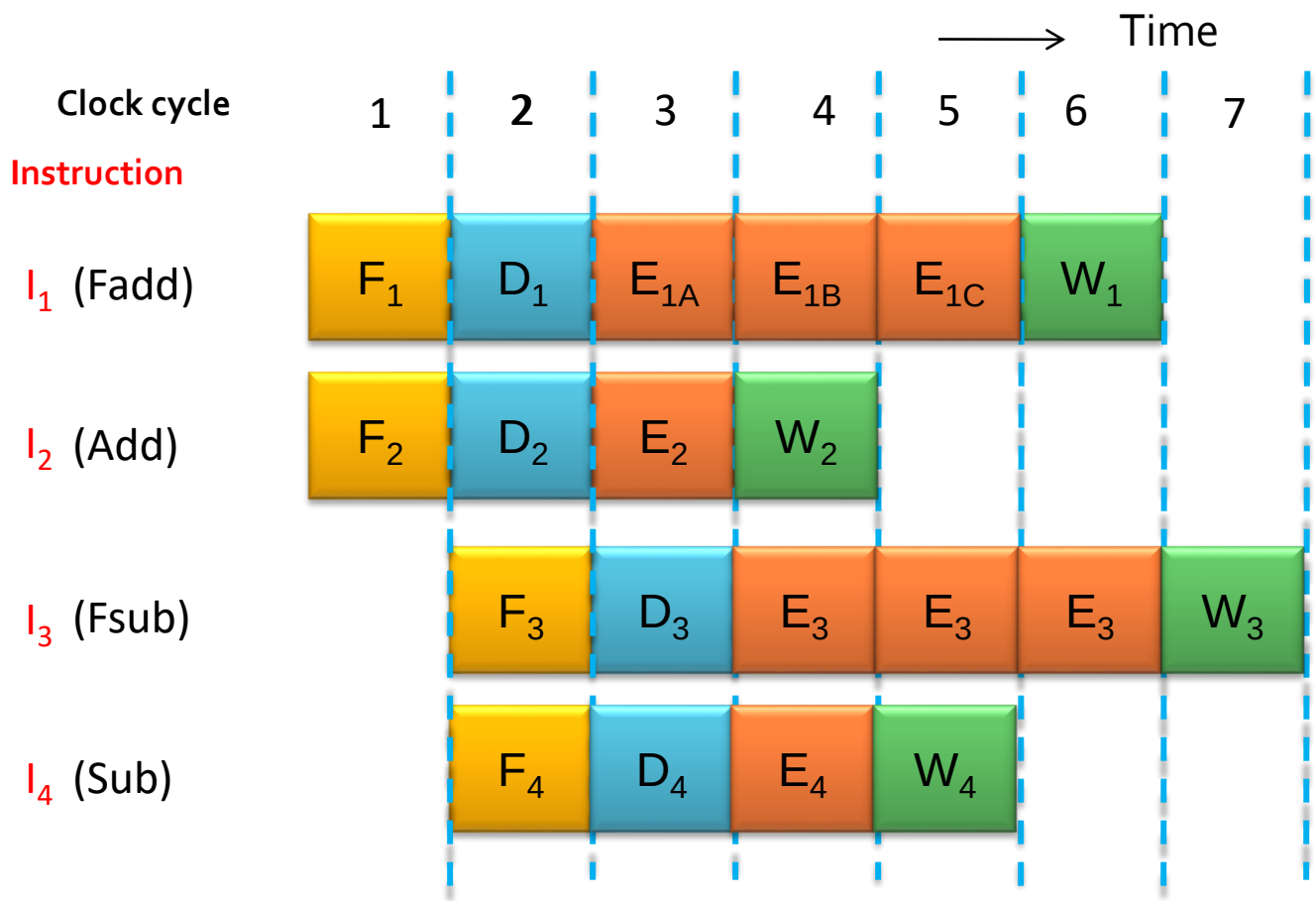
- The maximum throughput of a pipelined processor is one instruction per clock cycle.
- If we equip the processor with multiple processing units to handle several instructions in parallel in each processing stage, several instructions start execution in the same clock cycle – multiple-issue.
- Processors are capable of achieving an instruction execution throughput of more than one instruction per cycle – superscalar processors.
- Multiple-issue requires a wider path to the cache and multiple execution units.

Superscalar



A Processor with two execution units

Timing

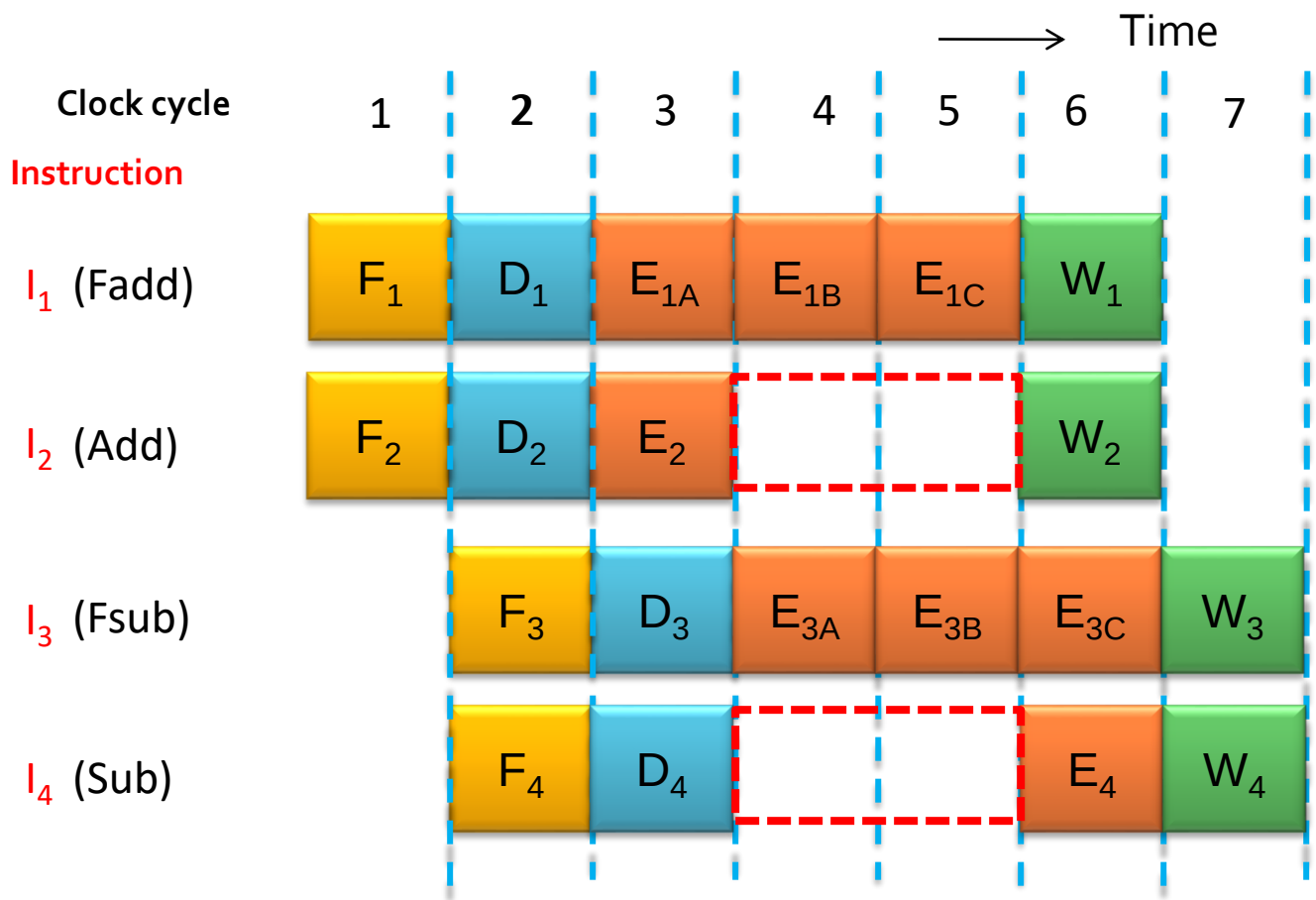


An example of instruction execution flow in the superscalar processor (assuming no hazards are present)

Out-of-Order Execution

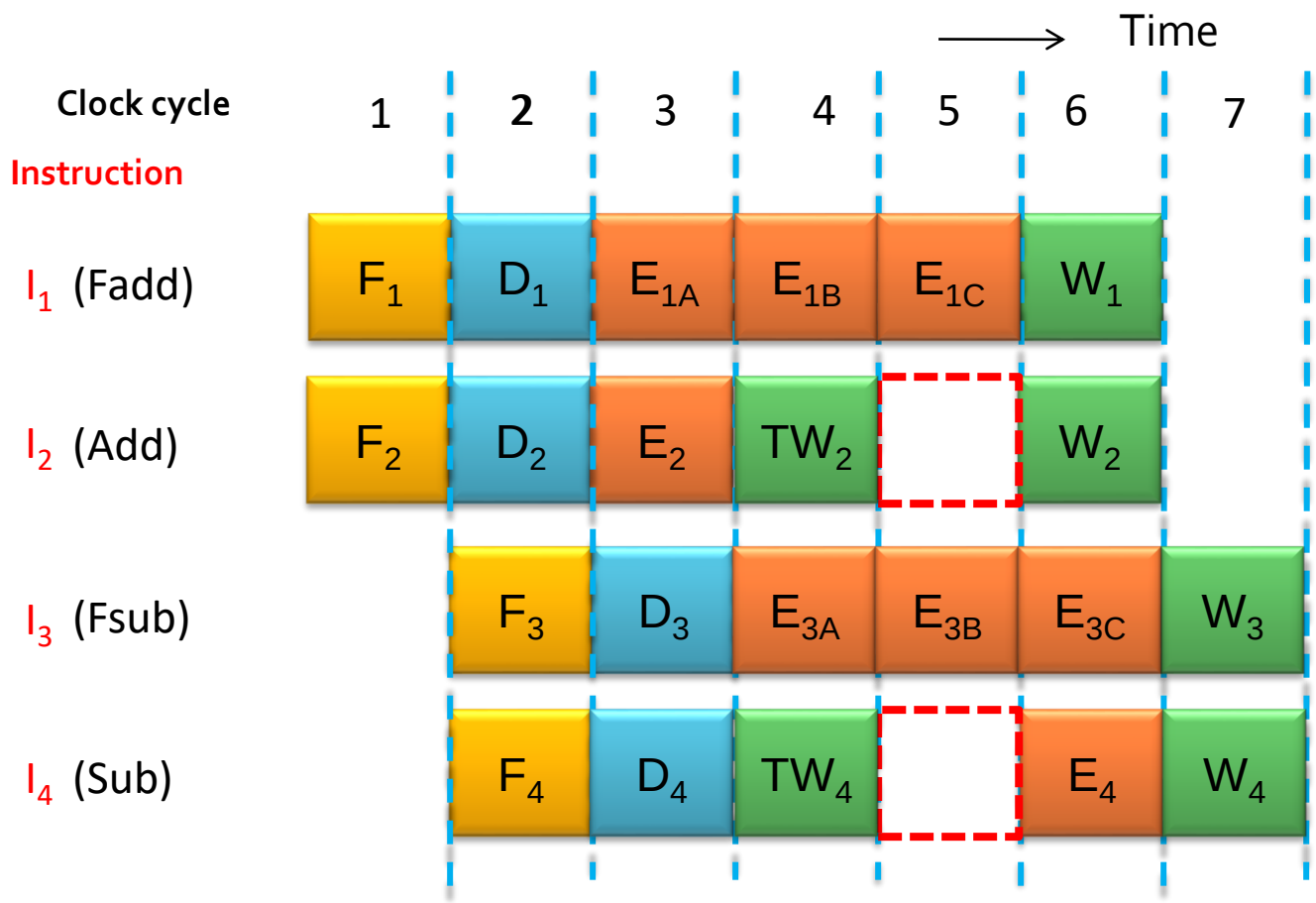
- Hazards
- Exceptions
- Imprecise exceptions
- Precise exceptions

Out-of-Order execution



Delayed Write

Out-of-Order execution



Using Temporary Registers

Performance Considerations

Performance Considerations

- The execution time T of a program that has a dynamic instruction count N is given by:

$$T = \frac{N \times S}{R}$$

where S is the average number of clock cycles it takes to fetch and execute one instruction, and R is the clock rate.

- Instruction throughput is defined as the number of instructions executed per second.

$$P_s = \frac{R}{S}$$

Performance Considerations

- An *n*-stage pipeline has the potential to increase the throughput by *n* times.
- However, the only real measure of performance is the total execution time of a program.
- Higher instruction throughput will not necessarily lead to higher performance.
- Two questions regarding pipelining
  - How much of this potential increase in instruction throughput can be realized in practice?
  - What is good value of *n*?

Number of Pipeline Stages

- Since an *n*-stage pipeline has the potential to increase the throughput by *n* times, how about we use a 10,000-stage pipeline?
- As the number of stages increase, the probability of the pipeline being stalled increases.
- The inherent delay in the basic operations increases.
- Hardware considerations (area, power, complexity,…)

Pipeline Speedup

- n=number of tasks to be performed
- Conventional Machine (Non-pipelined)
  - tn = clock cycle
  - t1 = time required to completed n tasks = n*tn
- Pipelined Machine (k-stages)
  - tp=clock cycle (time to complete each suboperation)
  - tk= time required to complete n tasks = (k+n-1)*tp
- Speedup
  - sk= speedup
  - sk= n*tn/ (k+n-1)*tp

Pipeline and Multiple Function Units

- Example
  - 4 stage pipeline
  - Sub operation in each stage; tp=20ns
  - 100 tasks to be executed
  - 1 task in non-pipeline system: 20*4 = 80ns
- Pipelined system
  - (k+n-1)*tp = (4+100-1)*20 = 2060ns
- Non-pipelined system
  - N*k*tp = 100 *4*20 =8000ns
- Speedup
  - sk = 8000/2060 = 3.88

Reference

- **Chapter 8: Computer Organization** by Carl Hamacher, Zvonko Vranesic and Safwat Zaky, 5 edition, McGraw Hill Publications.

Thank You!!